SC2017/CE2107 Microprocessor System Design and Development

Tutorial 1

Microprocessor Toolchain, ARM Cortex-M Assembly and Embedded C Programming

1. Below are snippets of the linker command file and map file of a CCS project.

Linker Command File

```
MEMORY
{
    MAIN       (RX) : origin = 0x00000000, length = 0x00040000
    INFO       (RX) : origin = 0x00200000, length = 0x00004000
    SRAM_CODE  (RWX): origin = 0x01000000, length = 0x00010000
    SRAM_DATA  (RW) : origin = 0x20000000, length = 0x00010000
}

SECTIONS
{
    .intvecs:   > 0x00000000
    .text    :   > MAIN
    .const   :   > MAIN
    .cinit   :   > MAIN
    .pinit   :   > MAIN
    .data    :   > SRAM_DATA
    .bss     :   > SRAM_DATA
    .sysmem  :   > SRAM_DATA
    .stack   :   > SRAM_DATA
}
```

Map File

```
MEMORY CONFIGURATION

         name            origin    length      used     unused   attr    fill
----------------------  --------  ---------  --------  --------  ----  --------
  MAIN                  00000000   00040000  0000087a  0003f786  R  X
  INFO                  00200000   00004000  00000000  00004000  R  X
  SRAM_CODE             01000000   00010000  000007c0  0000f840  RW X
  SRAM_DATA             20000000   00010000  000007c0  0000f840  RW
```

```
SECTION ALLOCATION MAP

 output                                  attributes/
section   page    origin      length       input sections
--------  ----  ----------  ----------   ----------------
.intvecs   0    00000000    000000e4
                  00000000    000000e4     startup_msp432p401r_ccs.obj (.intvecs:retain)

.binit     0    00000000    00000000

.text      0    000000e4    00000756
                  000000e4    0000032c     system_msp432p401r.obj (.text)
                  00000410    000000ec     SineFunction.obj (.text)
                  000004fc    0000009c     rtsv7M4_T_le_v4SPD16_eabi.lib : memcpy_t2.obj (.text)
                  00000598    0000007a                                   : memset_t2.obj (.text)
                  00000612    00000002                                   : mpu_init.obj (.text)
                  00000614    00000070                                   : autoinit.obj (.text)
                  00000684    00000068                                   : copy_decompress_lzss.obj (.text:decompress:lzss)
                  000006ec    00000054                                   : boot.obj (.text)
                  00000740    00000054                                   : exit.obj (.text)
                  00000794    0000004c                                   : cpy_tbl.obj (.text)
                  000007e0    00000018                                   : args_main.obj (.text)
                  000007f8    00000014                                   : _lock.obj (.text)
                  0000080c    0000000e                                   : copy_decompress_none.obj (.text:decompress:none)
                  0000081a    0000000e     startup_msp432p401r_ccs.obj (.text)
                  00000828    0000000c     rtsv7M4_T_le_v4SPD16_eabi.lib : copy_zero_init.obj (.text:decompress:ZI)
                  00000834    00000004                                   : pre_init.obj (.text)
                  00000838    00000002                                   : startup.obj (.text)
```

```
GLOBAL SYMBOLS: SORTED ALPHABETICALLY BY Name

address   name
-------   ----
00000827  ADC14_IRQHandler
00000827  AES256_IRQHandler
00000827  BusFault_Handler
00000741  C$$EXIT
00000827  COMP_E0_IRQHandler
00000827  COMP_E1_IRQHandler
00000827  CS_IRQHandler
00000411  CubicSin
```

a. If ‘MAIN’ memory corresponds to the on-chip flash memory of the processor, how much on-chip flash is free for future application expansion?

b. What is the total code size for the application?

c. Which file contribute to the largest code size to the application?

d. It appears that many Interrupt Handlers shared the same starting address of 0x827. Why is that so?

2. ARM Cortex-M processor supports the Thumb-2 ISA which consists of 16-bit and 32-bit instructions. What is the advantage of this approach, and why is it considered as a 32-bit processor design?

3. ARM processors design such as Cortex-M utilizes 32-bit memory mapped I/O for its I/O related devices and peripherals.
Explain the difficulty of such an approach faced in the design of the Thumb-2 (and A32 as well) ISA, and how it is resolved.

4. It is important to manage the memory usage in microprocessor-based systems, as the memory resources are typically rather limited due to cost and/or power considerations.
For the following C program snippet, identify where the various data will be located in the memory when the program is loaded into a processor system.

```
int a = 100;
int b;
char const c[4]="1234";

typedef struct
{ int d;
  char e;
} rec;

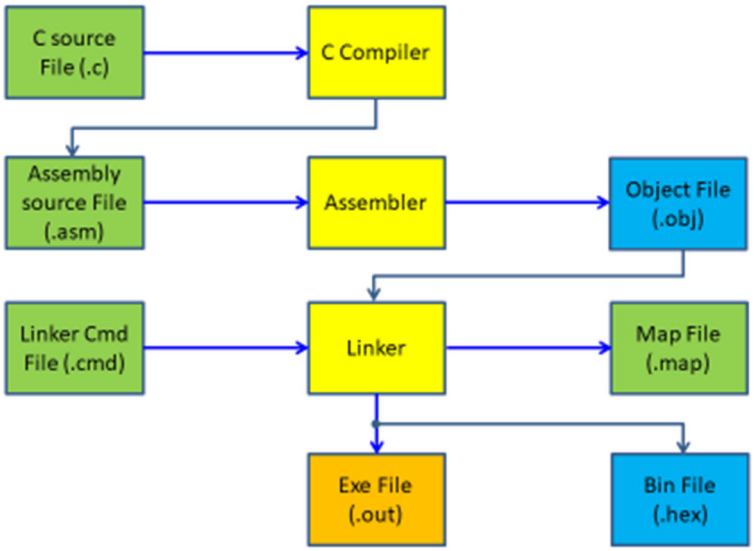

void main(void)
 {
   int x;
   rec *ptr_rec;

   ptr_rec = malloc (sizeof(rec));
       :
 }

 int func1(int arg1, arg2, char arg3, arg4, arg5)
 {
    int y;
    static int z = 0;
      :
 }
```

Optional

5. The diagram below shows the development toolchain of for embedded system programming.



a. What is the function of a Linker Command File?

b. What information is stored in the <.map> file? What is the difference between addresses stored in map file and listing file generated by the Assembler?

c. What information is stored in a hex file? What is the purpose of <.hex> file?

d. What is the difference between an executable file such as the <.out> file and the hex file?

e. What is the difference between the <.out> file generated by the Linker and the <.obj> file generated by the Assembler? Both seem to contain the machine code that made up the user program.

f. Can you spot any Modules/Components that is typically used when compiling a software project but is missing in the diagram?